The New York Public Library

Jail & Prison Services
40 West 20th Street
New York, NY 10011

DATE, 2024

FirstName LastName #12345678
Whatever Correctional Facility
P.O. Box 12345
New York, NY 12345

Dear (**Insert Name**),

Thank you for your letter asking for resources to support your technology class. You specified that you would like information on the history of Unix and Unix-like operating systems, such as Linux, with a focus on timelines, key players, and variants. You said you wanted “to be able to show how one invention in technology can lead to something with far reaching results”. You also asked for a primer on pseudocoding with keywords and instructions.

Please let us know if we can be of any assistance in the future. And, we've changed our address, please direct all future letters to Jail & Prison Services, 40 West 20th Street, New York, NY 10011.

Sincerely,
Jace
Jail and Prison Services
The New York Public Library

I. Origins and History of Unix and Unix-like Systems

Sourced from: http://ibgwww.colorado.edu/~lessem/psyc5112/usail/concepts/toc.html

The Unix operating system found its beginnings in **MULTICS**, which stands for Multiplexed Operating and Computing System. The MULTICS project began in the mid 1960s as a joint effort by **General Electric, Massachusetts Institute for Technology (MIT)** and **Bell Laboratories**. In 1969, Bell Laboratories pulled out of the project.

One of Bell Laboratories' people involved in the project was **Ken Thompson**. He liked the potential MULTICS had, but felt it was too complex and that the same thing could be done in a simpler way. In 1969, he wrote the first version of Unix, called UNICS. **UNICS** stands for Uniplexed Operating and Computing System. Although the operating system (OS) has changed, the name stuck and was eventually shortened to Unix.

Ken Thompson teamed up with **Dennis Ritchie**, who wrote the **first C compiler**. In 1973, they rewrote the Unix kernel in C. The following year, a version of Unix known as the Fifth Edition was first licensed to universities. The Seventh Edition, released in 1978, served as a dividing point for two divergent lines of Unix development. These two branches are known as **SVR4** (System V) and **BSD** (Berkeley Software Distribution).

Ken Thompson spent a year's sabbatical with the University of California at Berkeley. While there, he and two graduate students, **Bill Joy** and **Chuck Haley**, wrote the first Berkely version of Unix, which was distributed to students. This resulted in the source code being worked on and developed by many different people. The Berkeley version of Unix is known as BSD. From BSD came the **vi editor, C shell, virtual memory, Sendmail, and support for TCP/IP**.

For several years, SVR4 was more conservative, commercial, and well-supported. Today, SVR4 and BSD look very much alike. Probably the biggest cosmetic difference between them is the way the ps command functions.

The **Linux** OS was developed as a Unix look-alike and has a user command interface that resembles SVR4. (more on Linux in Section B)

i. The Connection Between Unix and C

At the time the first Unix was written, most operating systems developers believed that an OS must be written in an **assembly language** so that it could function effectively and gain access to

the hardware. Not only was Unix innovative as an OS, it was ground-breaking in that it was written in a language (C) that was **not an assembly language**.

The C language itself operates at a level that is just high enough to be portable to a variety of computer hardware. A great deal of publicly available Unix software is distributed as C programs that must be compiled before use. Many Unix programs follow C's syntax. **Unix system calls are regarded as C functions**. What this means for Unix system administrators is that an understanding of C can make Unix easier to understand.

ii. Why Use Unix?

One of the biggest reasons for using Unix is **networking capability**. With other OS, additional software must be purchased for networking. **With Unix, networking capability is simply part of the OS**. Unix is ideal for worldwide e-mail and connecting to the Internet.

Unix was founded on what could be called a "small is good" philosophy. **The idea is that each program is designed to do one job well**. Because Unix was developed by different people with different needs, it has grown into an OS that is both **flexible and easy to adapt** .

One of the unique things about Unix as an OS is that **it regards everything as a file**. Files can be divided into three categories; ordinary or plain files, directories, and special or device files.

- **Ordinary or plain files** in Unix are not all text files. They may also contain ASCII text, binary data, and program input or output. Executable binaries (programs) are also files, as are commands. When a user enters a command, the associated file is retrieved and executed. This is an important feature and contributes to the flexibility of Unix.

- **Directories** in Unix are properly known as directory files. They contain information such as owner, permissions, and size for a set of files.

- **Special files** are also known as device files. In Unix, all physical devices are accessed via device files; they are what programs use to communicate with hardware. Files hold information on location, type, and access mode for a specific device.

Unix was written in a **machine-independent language**. So Unix and Unix-like OS can run on a **variety of hardware**. These systems are available from many different sources, some of them at no cost. Because of this diversity and the ability to utilize the same "user-interface" on many different systems, **Unix is said to be an open system.**

A. Timeline of Unix Development

Sourced from: https://unix.org/what_is_unix/history_timeline.html

YEAR	EVENT	HISTORY
1969	**The Beginning**	The history of Unix starts back in 1969, when **Ken Thompson, Dennis Ritchie** and others started working on the "little-used PDP-7 in a corner" at Bell Labs and what was to become Unix.
1971	**First Edition**	It had an assembler for a PDP-11/20, file system, fork(), roff, and ed. It was used for text processing of patent documents.
1973	**Fourth Edition**	Unix rewritten in C. This made it portable and changed the history of OS's.
1975	**Sixth Edition**	Unix leaves home. Also widely known as Version 6, this is the first to be widely available outside of Bell Labs. The first BSD version (1.x) was derived from V6.
1979	**Seventh Edition**	Unix now had **C, UUCP,** and the **Bourne shell**. It was ported to the VAX and the kernel was more than 40 Kilobytes (K).
1980	**Xenix**	Microsoft introduces Xenix. 32V and 4BSD introduced.
1982	**System III**	**AT&T's Unix System Group (USG)** released System III, the first public release outside Bell Laboratories. SunOS 1.0 ships. HP-UX introduced. Ultrix-11 Introduced.
1983	**System V**	Computer Research Group (CRG), USG, and a third group merge to become **Unix System Development Lab**. AT&T announces Unix System V, the first supported release. Installed base 45,000.
1984	**4.2BSD**	University of California at Berkeley releases 4.2BSD, includes TCP/IP, new signals, and much more. **X/Open** formed.
1984	**SVR2**	System V Release 2 was introduced. At this time there are 100,000 Unix installations around the world.
1986	**4.3BSD**	4.3BSD released, including the internet name server. SVID introduced. NFS shipped. AIX announced. Installed base 250,000.
1987	**SVR3**	System V Release 3 including STREAMS, TLI, RFS. At this time there are

		750,000 Unix installations around the world. IRIX introduced.
1988		POSIX.1 published. **Open Software Foundation (OSF)** and Unix International (UI) were formed. Ultrix 4.2 ships.
1989		AT&T Unix Software Operation was formed in preparation for a spinoff of USL. Motif 1.0 ships.
1989	**SVR4**	Unix System V Release 4 ships, unifying System V, BSD, and Xenix. Installed base 1.2 million.
1990	**XPG3**	X/Open launches XPG3 Brand. OSF/1 debuts. Plan 9 from Bell Labs ships.
1991		**Unix System Laboratories (USL)** becomes a company - majority-owned by AT&T. **Linus Torvalds** commences Linux development. Solaris 1.0 debuts.
1992	**SVR4.2**	USL releases Unix System V Release 4.2 (Destiny). XPG4 Brand launched by X/Open. Novell announces intent to acquire USL. Solaris 2.0 ships.
1993	**4.4BSD**	4.4BSD is the final release from Berkeley. **Novell** acquires USL.
Late 1993	**SVR4.2MP**	Novell transfers rights to the "Unix" trademark and the Single Unix Specification to X/Open. COSE initiative delivers "Spec 1170" to X/Open for fast-track. Novell ships SVR4.2MP, the final USL OEM release of System V.
1994	**Single Unix Specification**	BSD 4.4-Lite eliminated all code claimed to infringe on USL/Novell. As the new owner of the Unix trademark, X/Open introduces the Single Unix Specification (formerly Spec 1170), separating the Unix trademark from any actual code stream.
1995	**Unix 95**	X/Open introduces the Unix 95 branding program for implementations of the Single Unix Specification. Novell sells UnixWare business line to SCO. Digital Unix was introduced. UnixWare 2.0 ships. OpenServer 5.0 debuts.
1996		**The Open Group** forms as a merger of OSF and X/Open.
1997	**Single Unix Specification, Version 2**	The Open Group introduces Version 2 of the Single Unix Specification, including support for real-time, threads and 64-bit and larger processors. The specification is made freely available on the web. IRIX 6.4, AIX 4.3 and HP-UX 11 ship.
1998	**Unix 98**	The Open Group introduces the Unix 98 family of brands, including Base,

		Workstation, and Server. First Unix 98 registered products shipped by Sun, IBM, and NCR. **The Open Source movement** starts to take off with announcements from Netscape and IBM. UnixWare 7 and IRIX 6.5 ship.
1999	**Unix at 30**	The Unix system reaches its 30th anniversary. Linux 2.2 kernel released. The Open Group and the IEEE commence joint development of a revision to POSIX and the Single Unix Specification. First LinuxWorld conferences. Tru64 Unix ships.
2001	**Single Unix Specification, Version 3**	Version 3 of the Single Unix Specification unites IEEE POSIX, The Open Group, and the industry efforts. Linux 2.4 kernel released. IT stocks face a hard time at the markets. The value of procurements for the Unix brand exceeds $25 billion. AIX 5L ships.
2003	**ISO/IEC 9945:2003**	The core volumes of Version 3 of the Single Unix Specification are approved as an international standard. The "Westwood" test suite ships for the Unix 03 brand. Solaris 9.0 E ships. Linux 2.6 kernel released.
2007		**Apple Mac OS X** certified to Unix 03.
2008	**ISO/IEC 9945:2008**	Latest revision of the Unix API set was formally standardized at ISO/IEC, IEEE, and The Open Group. Adds further APIs
2019	**Unix at 40**	IDC on Unix market -- says Unix $69 billion in 2008, predicts Unix $74 billion in 2013
2010	**Unix on the Desktop**	Apple reports 50 million desktops and growing -- these are Certified Unix systems.

B. Linux

Sourced from: https://www.geeksforgeeks.org/linux-history/

Linux was initially created by **Linus Torvalds** in 1991. At the time, Torvalds was a 21 year old computer science student at the University of Helsinki, Finland and began working on the Linux project as a personal endeavor. **The name Linux is a combination of his first name, Linus, and Unix, the operating system that inspired his projects**. At the time, most operating systems were proprietary and expensive. Torvalds wanted to create an operating system that was freely available to anyone who wanted to use the operating system. He originally released Linux as

free software under the **GNU General Public License**. This meant that anyone could use, modify, and redistribute his source code.

Early versions of Linux were primarily used by technology enthusiasts and software developers, but over time it has grown in popularity and is used in various types of devices such as **servers, smartphones, supercomputers, enterprise environments, and embedded systems**. Today, Linux is one of the most widely used operating systems in the world, with an estimated **2.76% of all desktop computers and more than 90% of the world's top supercomputers running on Linux,** and approximately **71.85% of all mobile devices run on Android**, which is Linux-based. The Linux community has expanded to include thousands of developers and users who work on the creation and upkeep of the operating system. Nowadays, Linux has many distributions (versions) namely: **Ubuntu, Fedora, Arch, Plasma, KDE, Mint, and Manjaro.**

i. How does Linux Work?

Linux was designed to be similar to Unix but evolved to run on hardware ranging from phones to supercomputers. All Linux-based operating systems include a Linux kernel that manages hardware resources and a set of software packages that make up the rest of the operating system. Organizations can also run Linux operating systems on Linux servers.

- **Kernel:** This is actually a component of the "Linux" system as a whole. The kernel, which controls the **CPU**, memory, and peripherals, serves as the brain of the system. The operating system's kernel is at the most fundamental level.

- **Desktop Environment**: The user actually engages in interaction at this point. There are numerous desktop environments available (GNOME, Cinnamon, Mate, Pantheon, Enlightenment, KDE, Xfce, etc.). Every desktop environment has pre-installed programmes (file managers, configuration tools, web browsers, games, etc.).

ii. Why Use Linux?

Linux is open-source software, meaning that the source code is freely available for anyone to use, modify, and distribute. This allows for a large and active community of developers to contribute to the development and maintenance of the operating system.

Linux is **highly customizable**, and users can easily install and configure different software packages to suit their needs. It is also known for its **stability and security**, as it is less prone to crashes and viruses than other operating systems.

iii. Events Leading to the Creation of Linux

Linux was heavily influenced by the Unix operating system. In the early 1980s, computer science professor **Andrew S. Tanenbaum** created a **small Unix-like operating system called Minix**. Minix was developed as an educational tool and the source code was made available to students. Linus Torvalds was one of those students. He was inspired by his Minix and used its source code as a starting point for his own projects. He also drew heavily on Unix design principles.

In September 1991, Linus released the first version of his Linux called **Linux 0.01**. It was a **command-line operating system** and was freely distributed on the Internet. In the years that followed, Linux quickly gained popularity among programmers and enthusiasts. A community of developers began to form around Linux, contributing to the development of the operating system by writing code, filing bug reports, and providing feedback.

In the late 1990s and early 2000s, the open-source nature of Linux made it **more flexible, cost-effective, and more secure than proprietary operating systems such as Windows**, making it a popular choice for enterprises and businesses. This increased acceptance led to the development of commercial support and services for Linux.

As Linux became more popular, various groups of developers began creating their own versions of the operating system, called **distributions**. Some of the most popular distributions are **Red Hat, Debian, and Ubuntu**. These distros **contain the Linux kernel** and a number of its packages of easy-to-use tools and software that make using Linux easy for both developers and end users.

With the growth of cloud computing and the **Internet of Things**, Linux continues to gain traction in the enterprise. Linux is now widely used as an operating system for servers, mainframes, and supercomputers. It's also used in embedded systems, mobile devices, smart TVs, and other consumer electronics.

iv. Official Mascot of Linux

The official Linux mascot is a penguin named **Tux**. Created by artist **Larry Ewing** in 1996, Tux quickly became popular in the Linux community and is now one of Linux's most recognizable icons. Tux was chosen as the mascot because

the penguin is a rare animal found in the wild in Antarctica, just as Linux is a unique and powerful operating system. The name Tux comes from the short form “Torvalds Unix“, in honor of the creator of Linux, Linus Torvalds.

v. Development of Linux

The Linux ecosystem is a constantly evolving and expanding platform, so there is a lot of development going on. Notable recent developments include:

Linux 5.11 kernel release: It includes new features such as AMD Zen 3 processor support, memory management system improvements, and new hardware support.

Continued development of various Linux distributions: Ubuntu 21.04 released in April 2021. It features an updated Gnome desktop environment, improved ZFS file system support, and new security features.

Development of new open-source software and tools for Linux: The release of version 6.0 of Ansible automation tools brings new features such as support for Windows Subsystem for Linux 2 (WSL2) and improved support for Kubernetes.

The rise of containerization and orchestration technologies such as Docker and Kubernetes: They are becoming more and more common in deploying and managing Linux-based applications.

C. Popular Unix/Linux Distributions

Sourced from: https://distrowatch.com/dwres.php?resource=major

“Popular” is subjective, distributions vary in usability depending on personal need, but as a general rule, all of these are popular and have very active forums or mailing lists where you can ask questions if you get stuck.

Debian

Debian GNU/Linux was first announced in 1993. Its founder, **Ian Murdock**, envisaged the creation of a completely non-commercial project developed by hundreds of volunteer developers in their spare time. It is responsible for inspiring over 120 Debian-based distributions and live CDs. The actual development of Debian takes place in three main branches of increasing levels of stability: "Unstable" (also known as "Sid"), "Testing", and "Stable". This progressive

integration and stabilization of packages and features, together with the project's well-established quality control mechanisms, has earned Debian its reputation of being one of the best-tested and most bug-free distributions available today.

However, this lengthy and complex development style also has some drawbacks: the stable releases of Debian are not particularly up-to-date and they age rapidly, especially since new stable releases are only published once every 2-3 years. Those users who prefer the latest packages and technologies are forced to use the potentially buggy Debian Testing or Unstable branches. The highly democratic structures of Debian have led to controversial decisions and gives rise to infighting among the developers. This has contributed to stagnation and reluctance to make radical decisions that would take the project forward.

Ubuntu

Since its announcementin September of 2004, Ubuntu grew to become the most popular desktop Linux distribution and has greatly contributed towards developing an easy-to-use and free desktop operating system that can compete well with any proprietary ones available on the market.

What was the reason for Ubuntu's stunning success? Firstly, the project was created by **Mark Shuttleworth**, a South African multimillionaire, a former **Debian** developer, and the world's second space tourist, whose company, the Isle of Man-based Canonical Ltd, is currently financing the project. Ubuntu is based on **Debian "Sid"** (Debian's Unstable branch). Secondly, Ubuntu had learned from the mistakes of other similar projects and avoided them from the start - it created an excellent web-based infrastructure with a Wiki-style documentation, creative bug-reporting facility, and professional approach to the end users. And thirdly, thanks to its wealthy founder, Ubuntu was able to ship free CDs to all interested users, thus contributing to the rapid spread of the distribution.

MX Linux

MEPIS Linux was a **Debian**-based desktop Linux distribution designed for both personal and business purposes. It included (for the time) cutting-edge features such as a live, installation and recovery CD, automatic hardware configuration, NTFS partition resizing, ACPI power management, WiFi support, anti-aliased TrueType fonts, and a personal firewall. Though MEPIS Linux was eventually discontinued, its community continued on and merged technology from MEPIS with the very lightweight, Debian-based antiX distribution. The result is a project called MX Linux. MX Linux is based on Debian's Stable branch and features components developed by the MEPIS and antiX communities.

Linux Mint

A distribution based on **Ubuntu**, was first launched in 2006 by **Clement Lefebvre**, a French-born IT specialist living in Ireland. Originally maintaining a Linux web site dedicated to providing help, tips and documentation to new Linux users, the author saw the potential of developing a Linux distribution that would address the many usability drawbacks associated with the generally more technical, mainstream products. After soliciting feedback from the visitors on his web site, he proceeded with building what many refer to today as an "improved Ubuntu" or "Ubuntu done right". The developers have been adding a variety of graphical "mint" tools for enhanced usability; this includes mintDesktop - a utility for configuring the desktop environment, mintMenu - a new and elegant menu structure for easier navigation, mintInstall - an easy-to-use software installer, and mintUpdate - a software updater, just to mention a few.

Fedora

Although Fedora was formally unveiled only in September 2004, its origins effectively date back to 1995 when it was launched by two Linux visionaries -- **Bob Young** and **Marc Ewing** -- under the name of **Red Hat Linux**. The company's first product, Red Hat Linux 1.0 "Mother's Day", was released in the same year and was quickly followed by several bug-fix updates. In 1997, Red Hat introduced its revolutionary RPM package management system with dependency resolution and other advanced features which greatly contributed to the distribution's rapid rise in popularity.

In 2003, just after the release of Red Hat Linux 9, the company introduced some radical changes to its product line-up. It retained the Red Hat trademark for its commercial products, notably Red Hat Enterprise Linux, and introduced Fedora Core (later renamed to Fedora), a Red Hat-sponsored, but community-oriented distribution designed for the "Linux hobbyist".

Although Fedora's direction is still largely controlled by Red Hat, Inc. and the product is sometimes seen -- rightly or wrongly -- as a test bed for Red Hat Enterprise Linux, there is no denying that Fedora is one of the most innovative distributions available today. Its contributions to the Linux kernel, glibc and GCC are well-known and its integration of SELinux functionality, virtualisation technologies, systemd service manager, cutting-edge journaled file systems, and other enterprise-level features are much appreciated among the company's customers. On a negative side, Fedora still lacks a clear desktop-oriented strategy that would make the product easier to use for those beyond the "Linux hobbyist" target.

FreeBSD

FreeBSD, an indirect descendant of **AT&T UNIX** via the **Berkeley Software Distribution (BSD**), has a long and turbulent history dating back to 1993. Unlike Linux distributions, which are defined as integrated software solutions consisting of the Linux kernel and thousands of software applications, FreeBSD is a tightly integrated operating system built from a BSD kernel and the so-called "userland" (therefore usable even without extra applications). This distinction is largely lost once installed on an average computer system - like many Linux distributions, a large collection of easily installed, (mostly) open source applications are available for extending the FreeBSD core, but these are usually provided by third-party contributors and aren't strictly part of FreeBSD.

FreeBSD has developed a reputation for being a fast, high-performance and extremely stable operating system, especially suitable for web serving and similar tasks. Many large web search engines and organisations with mission-critical computing infrastructures have deployed and used FreeBSD on their computer systems for years. Compared to Linux, FreeBSD is distributed under a much less restrictive license, which allows virtually unrestricted re-use and modification of the source code for any purpose. Even parts of **Apple's macOS** are known to have been derived from FreeBSD. Besides the core operating system, the project also provides over 24,000 software applications (called ports) in binary and source code forms for easy installation on top of the core FreeBSD.

While FreeBSD can certainly be used as a desktop operating system, it doesn't compare well with popular Linux distributions in this department. The text-mode system installer offers little in terms of hardware detection or system configuration, leaving much of the dirty work to the user in a post-installation setup. In terms of support for modern hardware, FreeBSD generally lags behind Linux, especially in supporting cutting-edge desktop and laptop gadgets, such as wireless network cards or digital cameras. Those users seeking to exploit the speed and stability

of FreeBSD on a desktop or workstation should consider one of the available desktop FreeBSD projects, rather than FreeBSD itself.

openSUSE

The beginnings of openSUSE date back to 1992 when four German Linux enthusiasts -- Roland Dyroff, Thomas Fehr, Hubert Mantel and Burchard Steinbild -- launched the project under the name of SuSE (Software und System Entwicklung) Linux. In the early days, the young company sold sets of floppy disks containing a German edition of Slackware Linux, but it wasn't long before SuSE Linux became an independent distribution with the launch of version 4.2 in May 1996. In the following years, the developers adopted the RPM package management format and introduced YaST, an easy-to-use graphical system administration tool. Frequent releases, excellent printed documentation, and easy availability of SuSE Linux in stores across Europe and North America resulted in growing popularity for the distribution.

Arch Linux

The KISS (keep it simple, stupid) philosophy of Arch Linux was devised around the year 2002 by **Judd Vinet**, a Canadian computer science graduate who launched the distribution in the same year. For several years it lived as a marginal project designed for intermediate and advanced Linux users and only shot to stardom when it began promoting itself as a "rolling-release" distribution that only needs to be installed once and which is then kept up-to-date thanks to its powerful package manager and an always fresh software repository. As a result, Arch Linux "releases" are simply monthly snapshots of the install media.

Besides featuring the much-loved "rolling-release" update mechanism, Arch Linux is also renowned for its fast and powerful package manager called "Pacman", the ability to install software packages from source code, easy creation of binary packages thanks to its AUR infrastructure, and the ever increasing software repository of well-tested packages. Its highly-regarded documentation, complemented by the excellent Arch Linux Handbook, makes it possible for even less experienced Linux users to install and customise the distribution. The powerful tools available at the user's disposal mean that the distro is infinitely customisable to the most minute detail and that no two installations can possibly be the same.

On the negative side, any rolling-release update mechanism has its dangers: a human mistake creeps in, a library or dependency goes missing, a new version of an application already in the repository has a yet-to-be-reported critical bug... It's not unheard of to end up with an unbootable system following a Pacman upgrade. As such, Arch Linux is a kind of distribution

that requires its users to be alert and to have enough knowledge to fix any such possible problems.

Manjaro

Manjaro is a Linux-based open-source operating system that does not entail users to pay a fee or display ads. It esteems their privacy and permits them total control over how their hardware is set up and operated. Relying on your needs, you can launch it on tablets, mobiles, PCs, laptops and boards, as well as for gaming and 3D applications. A widespread and extremely customizable Linux operating system, Manjaro is predicated on **Arch Linux.** With its hardware detection manager, Manjaro Linux delivers excellent hardware support. A key focus of Manjaro Linux is to be accessible and user-friendly. The kernel can easily be modified without mandating any intricate troubleshooting. Furthermore, Arch Linux-based distributions allow you to choose your own components. It can be customized to match your laptop.

II. Pseudocoding

Sourced from: https://builtin.com/data-science/pseudocode

Pseudocoding in data science or web development is a technique used to **describe the distinct steps of an algorithm** in a way that's easy for anyone with basic programming knowledge to understand. In the initial state of solving a problem, it helps to eliminate the limitations offered by a specific programming language's **syntax rules** when we design or validate an algorithm. By doing this, we can focus our attention on the thought process behind the algorithm and how it will (or won't) work instead of focusing on our syntax.

Although **pseudocode is a syntax-free description of an algorithm,** it must provide a full description of the algorithm's logic so that moving from pseudocode to implementation is merely a task of translating each line into code using the syntax of any given programming language.

A. Why Use Pseudocode?

Pseudocode is an underestimated and under-utilized tool within the programming community, but a clear, concise, straightforward pseudocode can make a big difference on the road from idea to implementation — and a much smoother ride for the programmer.

Pseudocode Is Easier to Read

Often, programmers work alongside people from other fields such as mathematicians, managers and business partners. Using pseudocode to explain the mechanics of the code makes communicating between different specialties easier and more efficient.

Pseudocode Simplifies Code Construction
When the programmer goes through the process of developing and generating pseudocode, converting that into real code written in any programming language will become much easier and faster.

Pseudocode Is a Helpful Starting Point for Documentation
Documentation is an essential aspect of building a good project but starting documentation is often the most difficult part of the process. Pseudocode can represent a good starting point for what the documentation should include. Sometimes, programmers even include the pseudocode as a docstring at the beginning of the code file.

Pseudocode Allows for Quick Bug Detection
Since pseudocode is written in a human-readable format, it is easier to edit and discover bugs before actually writing a single line of code. We can edit pseudocode more efficiently than testing, debugging and fixing actual code.

B. How to Write Pseudocode

When writing pseudocode, everyone has their own style of presenting since humans are reading it and not a computer; pseudocode's rules are less rigorous than those of a programming language. However, there are some simple rules that help make pseudocode more universally understood.

1. Always capitalize the initial word (often one of the main six constructs)
2. Keep each piece of your pseudocode in the order in which it needs to be executed
3. Make only one statement per line
4. Indent to show hierarchy, improve readability and show nested constructs
5. Always end multi-line sections using any of the END keywords (ENDIF, ENDWHILE, etc.)
6. Keep your statements programming language independent

7. Use the naming domain of the problem, not that of the implementation. For instance: “Append the last name to the first name” instead of “name = first+last”
8. Pseudocode does not typically use variables, instead, describe what the program should do with variables and data (account numbers, names, transaction amounts, etc.)
9. Keep it simple, concise and readable

ii. The Main Constructs of Pseudocode

At its core, pseudocode is the ability to represent six programming constructs (always written in uppercase): **SEQUENCE, CASE, WHILE, REPEAT-UNTIL, FOR,** and **IF-THEN-ELSE**. These constructs — also called keywords — are used to describe the control flow of the algorithm.

- **SEQUENCE** represents **linear tasks** sequentially performed one after the other.
- **WHILE** is a **loop** with a condition at its beginning.
- **REPEAT-UNTIL** is a loop with a condition at the bottom.
- **FOR** is another way of looping.
- **IF-THEN-ELSE** is a **conditional statement** changing the flow of the algorithm.
- **CASE** is the generalization form of IF-THEN-ELSE.

Of course, based on the field you’re working in, you might add more constructs (keywords) to your pseudocode glossary as long as you never use these keywords as variable names and ensure they’re well-known within your field or company. Pseudocode is subjective and not standardized. You might even want to get rid of any coding commands altogether and just define each line's process in plain language. For example, "IF input is odd, OUTPUT 'Y'" might become "IF user enters an odd number, DISPLAY 'Y'".

IF condition THEN instruction — This means that a given instruction will only be conducted if a given condition is true. "Instruction", in this case, means a step that the program will perform, while "condition" means that the data must meet a certain set of criteria before the program takes action.

WHILE condition DO instruction — This means that the instruction should be repeated again and again until the condition is no longer true.

DO instruction WHILE condition — This is very similar to "WHILE condition DO instruction". In the first case, the condition is checked before the instruction is conducted, but in the second case the instruction will be conducted first; thus, in the second case, theinstruction will be conducted at least one time.

FUNCTION (arguements): instruction — This means that every time a certain function is used in the code, it is an abbreviation for a certain instruction. "Arguments" are lists of variables that you can use to clarify the instruction.

iv. Examples

Following these rules helps you generate readable pseudocode. Say an employer is designing a 10-question, multiple-choice quiz for employees to take after reading content on workplace safety. Those who get at least eight questions correct pass the quiz and receive a certificate of completion. Those who don't reach this benchmark must retake the quiz. The pseudocode for an algorithm determining those who pass and fail could look like this:

- **IF** employee gets eight or more questions correct
    - Display message: "Congratulations on passing the quiz!"
    - Transition to printable certificate of completion page
- **ELSE**
    - Display message: "Let's try again!"
    - Transition back to first page of quiz

Consider an online store that offers discounts on orders of $50 or more. The pseudocode for an algorithm determining if a user gets a discount could go as follows:

- **IF** price of an order is equal to or greater than $50
    - Apply discount
    - Display message: "Discount has been applied!"
- **ELSE**
    - Do not apply discount
    - Display message: "Add more items to cart for discount!"

C. Pseudocode Glossary

Algorithm: a set of mathematical instructions or rules that, especially if given to a computer, will help to calculate an answer to a problem

Construct: keywords that represent the flow of control in an algorithm

Function: a sequence of commands that can be reused together later in a program

Pseudocode: a notation resembling a simplified programming language, used in program design

Syntax: the rules that control the structure of the symbols, punctuation, and words of a programming language. Grammar for coding.